I S P 2 5 0 · S E M E S T E R 4 · 2 0 2 4

Simon Says!

A colour-sequence memory game built with MIT App Inventor

Prepared by

Adam Iskandar bin Nazri

Student ID: 2024277906

Diploma in Computer Science · UiTM Jasin

Course: ISP250 — Introduction to Software Programming

1. PROJECT OVERVIEW

Simon Says! is a fully functional mobile memory game developed for Android using MIT App Inventor, a block-based visual programming environment. Inspired by the classic Simon electronic game, players must memorise and repeat a growing sequence of colour flashes. The application was built as the final project for ISP250 (Introduction to Software Programming) during Semester 4 at UiTM Jasin.

Platform	Android (MIT App Inventor)
Tool	MIT App Inventor 2 (Block-based programming)
Screens	6 — MainMenu, GameScreen, GameOverScreen, DifficultyScreen, HighScoresScreen, InstructionsScreen
Data Storage	TinyDB (on-device persistent storage)
Course	ISP250 — Introduction to Software Programming, Semester 4, 2026

2. APP SCREENS

The application consists of six screens, each serving a distinct purpose in the user journey.

Main menu

Navigation hub with Play, Difficulty, High Scores and How to Play. Background music plays and resumes on return.

Gameplay

Four colour tiles flash in sequence. Player taps them in order. Score increments each round.

Difficulty

Step-by-step rules for first-time players. Back to Menu button to return.

Leaderboard

Three speed settings: Easy (800 ms), Medium (600 ms), Hard (350 ms). Current selection shown live.

Instructions

Top 5 scores stored in TinyDB. Clear All button with confirmation dialog.

Game over

Displays score and all-time best. New record flagged. Play Again or Main Menu options.

3. KEY FEATURES

Six core systems designed and implemented from scratch.

3-level difficulty Flash speed stored in TinyDB: Easy 800 ms, Medium 600 ms, Hard 350 ms. Persists across app restarts.	**Persistent leaderboard** Top 5 scores sorted with insertion sort and stored in TinyDB. New all-time bests detected.
Sound & music Background music on menu and game screens. Beep on each colour flash. Buzz on wrong input.	**Pause & resume** Full pause overlay. Both Clock timers and music halt and restore cleanly on resume.
Input validation isFlashing flag blocks colour taps during sequence display, preventing false/early inputs.	**Instant replay** Play Again sends 'restart' to GameScreen via OtherScreenClosed, resetting state immediately.

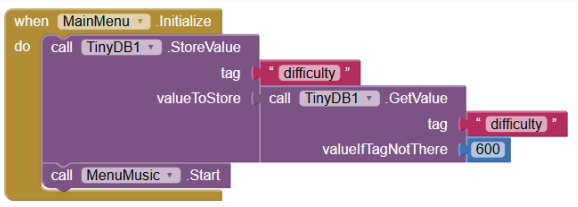
4. SKILLS DEMONSTRATED

- TinyDB persistence — saving and reading difficulty settings and high score lists across sessions
- Clock-based animation — dual-clock system (Clock1 for flash timing, Clock2 for colour reset)
- Insertion sort algorithm — sorting new scores into the stored top-5 list in GameOverScreen
- Multi-screen navigation — open/close screen with value passing (result: restart / menu)
- Audio management — playing, stopping, and pausing background music and sound effects
- Global state variables — sequence, score, playerIndex, flashIndex, isFlashing, difficulty
- List manipulation — add items, select item, length of list, create empty list
- Event-driven programming — button click, screen initialize, clock timer, other screen closed

5. BLOCK CODE — BLOCK BY BLOCK

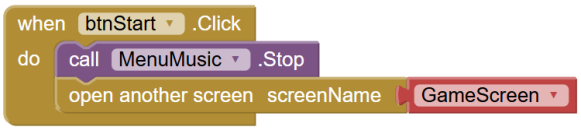
Each individual block from all six screens is shown below with its own explanation. Blocks are grouped by screen.

MainMenu Screen



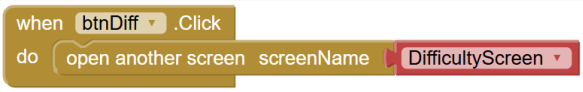
MainMenu.Initialize

When the Main Menu screen starts, the difficulty setting is loaded from TinyDB (defaulting to 600 ms if not yet set) and immediately stored back to ensure the tag always holds a value. The menu background music is then started.



btnStart.Click

When the Play Game button is tapped, the menu music stops and the app opens the GameScreen to begin a new session.



btnDiff.Click

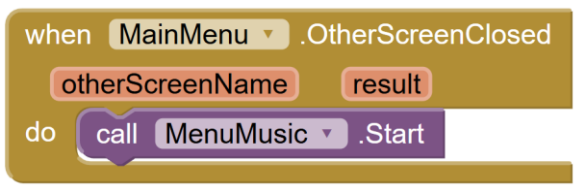
When the Difficulty button is tapped, the DifficultyScreen opens so the player can change the flash-speed setting.

btnScore.Click

When the High Scores button is tapped, the HighScoresScreen opens to display the top-5 leaderboard.

btnInstruct.Click

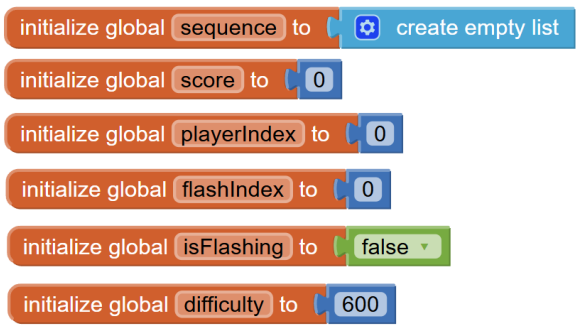
When the How to Play button is tapped, the InstructionsScreen opens to show the game rules.



MainMenu.OtherScreenClosed

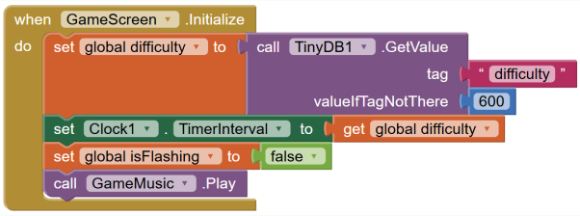
Whenever the player returns from any secondary screen (Difficulty, Scores, Instructions), this event fires and the menu music automatically resumes playing.

GameScreen



Global Variables

Six global variables are declared at startup: sequence (the list of colour numbers to flash), score, playerIndex (which colour the player must tap next), flashIndex (which colour is currently being shown during the flash phase), isFlashing (true during animation — blocks all player input), and difficulty (the timer interval in ms from TinyDB).

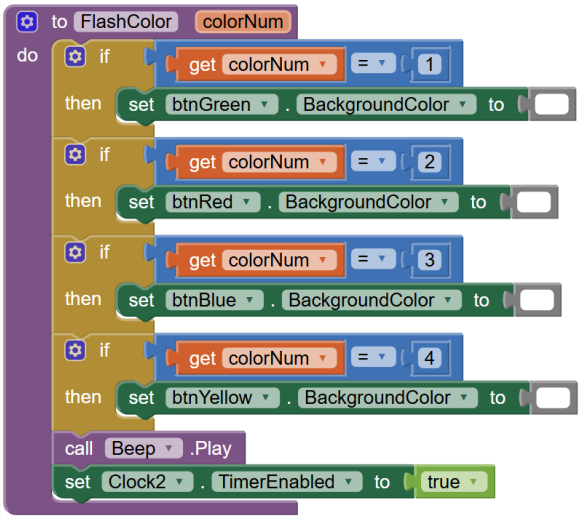


GameScreen.Initialize

When the GameScreen loads, the difficulty value is read from TinyDB and applied to Clock1's TimerInterval. isFlashing is reset to false and the game background music starts playing.

Procedure: AddNextColor

Starts a new round. Resets playerIndex and flashIndex to 0, sets isFlashing to true to block player taps, adds a random integer 1–4 to the sequence list, updates the status label to "Watch carefully!" and enables Clock1 to begin the flash animation.



Procedure: FlashColor (colorNum)

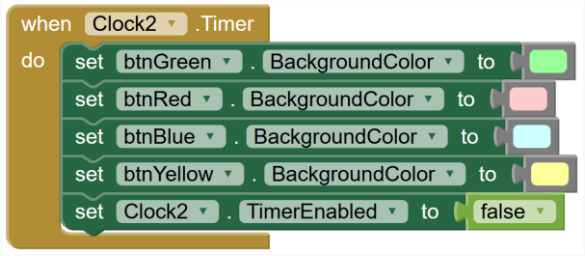
Called by Clock1 each tick to highlight one colour tile. Receives a colour number (1=Green, 2=Red, 3=Blue, 4=Yellow), sets that button's background to white, plays a beep sound, and enables Clock2 to restore the colour after a short delay.

Procedure: CheckInput (tappedColor)

Called when a colour button is tapped. Increments playerIndex by 1 then checks if the tapped colour matches the sequence at that position. If correct and the player has finished the full sequence: score increments, AddNextColor is called for the next round. If wrong: a buzz plays, music stops, the status changes to "Wrong! Game over", and the GameOverScreen opens with the current score passed as the start value.

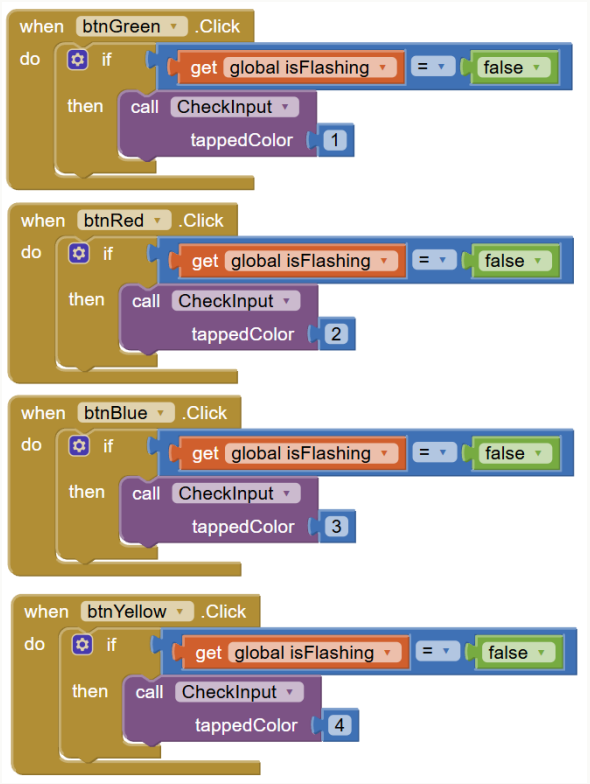
Clock1.Timer — Flash Animation Driver

Fires at the interval set by the difficulty value. Each tick advances flashIndex. If there are still colours left in the sequence to show, FlashColor is called with the next colour number. Once all colours have been flashed, Clock1 stops, isFlashing is set to false, and the status label changes to "Your turn!" so the player knows they can start tapping.



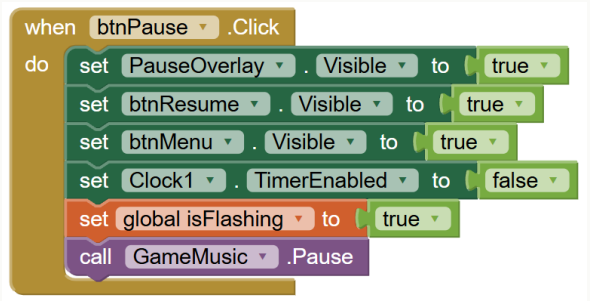
Clock2.Timer — Colour Reset

Fires briefly after FlashColor sets a tile to white. Resets all four colour buttons back to their normal colours (green, red, blue, yellow) and immediately disables itself. This creates the flash-on / flash-off visual effect for each tile in the sequence.



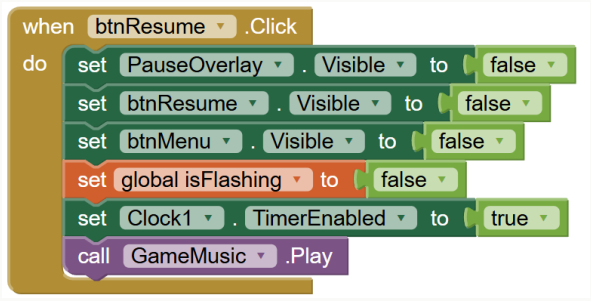
Colour Button Handlers (Green, Red, Blue, Yellow)

Each of the four colour buttons first checks isFlashing — if the animation is still running, the tap is completely ignored. If the player is allowed to tap, CheckInput is called with the button's colour number: 1 for Green, 2 for Red, 3 for Blue, 4 for Yellow.



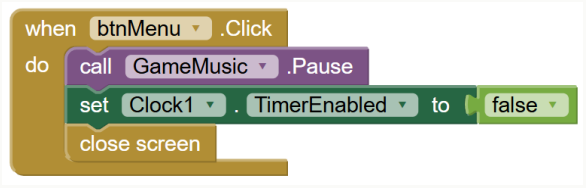
btnPause.Click

Shows the pause overlay panel and makes the Resume and Menu buttons visible. Disables Clock1 to stop the animation, sets isFlashing to true to block colour taps, and pauses the background music.



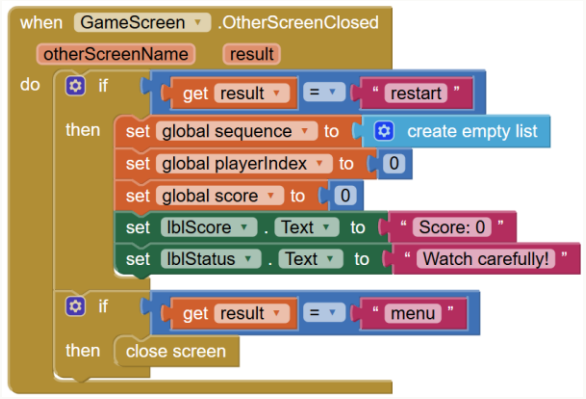
btnResume.Click

Hides the pause overlay and the Resume/Menu buttons. Re-enables Clock1 to resume the animation, resets isFlashing to false, and resumes the background music from where it was paused.



btnMenu.Click

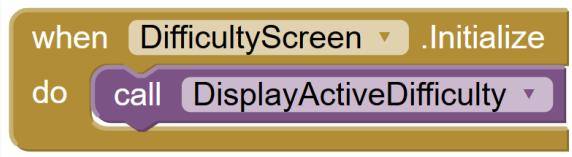
Pauses the background music and disables Clock1, then closes the GameScreen to return to the Main Menu.



GameScreen.OtherScreenClosed

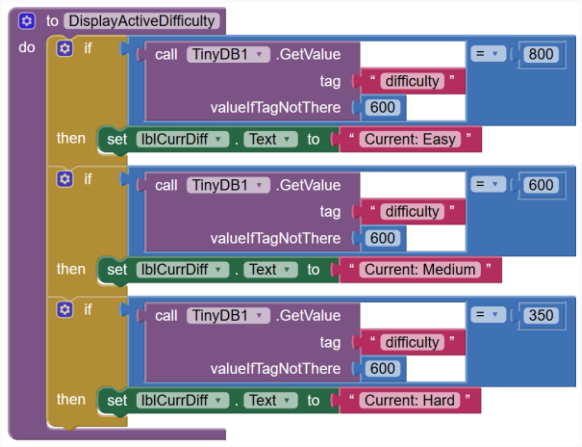
Fires when GameOverScreen closes and passes a result back. If the result is "restart": all game state is reset (sequence cleared, playerIndex/flashIndex/score zeroed, labels reset to their defaults) and AddNextColor is called to immediately begin a new game. If the result is "menu": the GameScreen closes and the player returns to the Main Menu.

DifficultyScreen



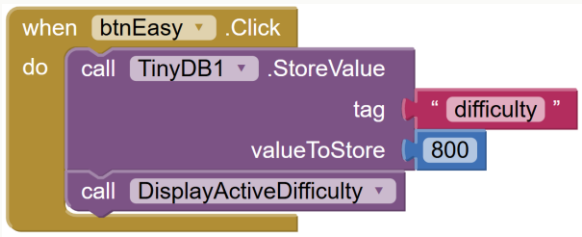
DifficultyScreen.Initialize

When the Difficulty screen opens, DisplayActiveDifficulty is called immediately to show the player which difficulty is currently active before they make any change.



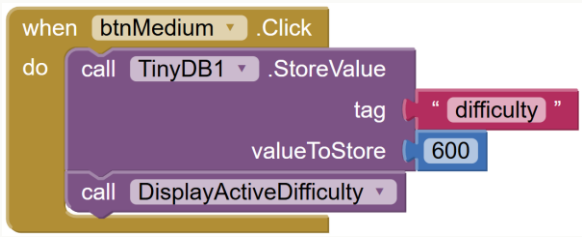
Procedure: DisplayActiveDifficulty

Reads the difficulty tag from TinyDB (defaulting to 600 if not set). Compares the value: 800 → sets the label to "Current: Easy", 600 → "Current: Medium", 350 → "Current: Hard". This gives instant visual feedback of the active setting after every button tap.



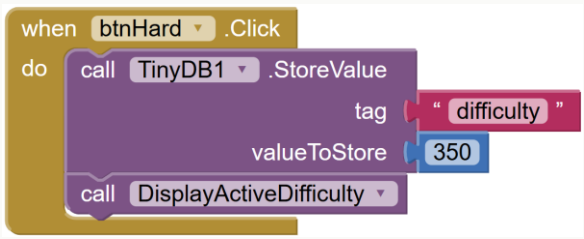
btnEasy.Click

Stores 800 (ms) as the difficulty value in TinyDB under the "difficulty" tag, then calls DisplayActiveDifficulty to immediately update the label to "Current: Easy".



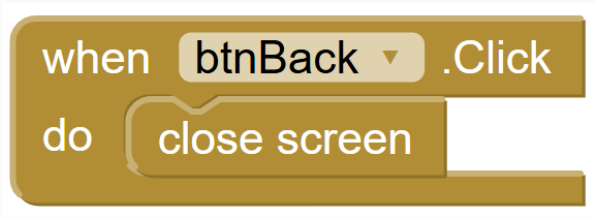
btnMedium.Click

Stores 600 (ms) as the difficulty value in TinyDB, then calls DisplayActiveDifficulty to update the label to "Current: Medium". This is also the default speed.



btnHard.Click

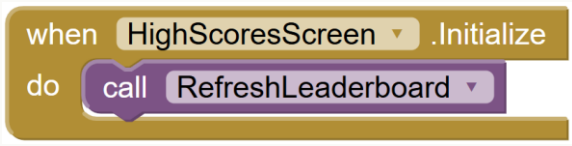
Stores 350 (ms) as the difficulty value in TinyDB, then calls DisplayActiveDifficulty to update the label to "Current: Hard". At this speed the tiles flash almost twice as fast as Easy.



btnBack.Click

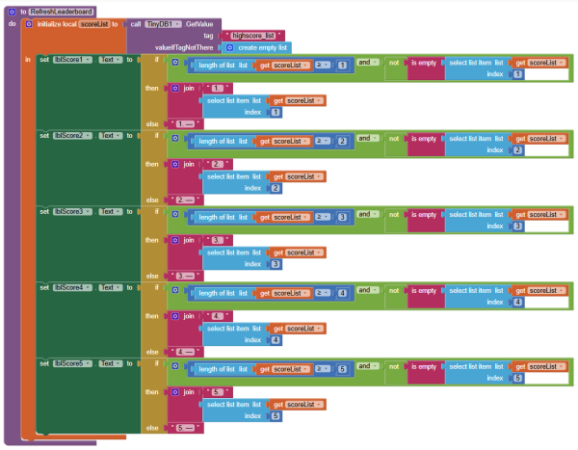
Closes the DifficultyScreen and returns to the Main Menu. The chosen difficulty is already saved in TinyDB so it takes effect the next time GameScreen loads.

HighScoresScreen



HighScoresScreen.Initialize

When the screen opens, RefreshLeaderboard is called immediately to populate the five score labels from the stored TinyDB list.

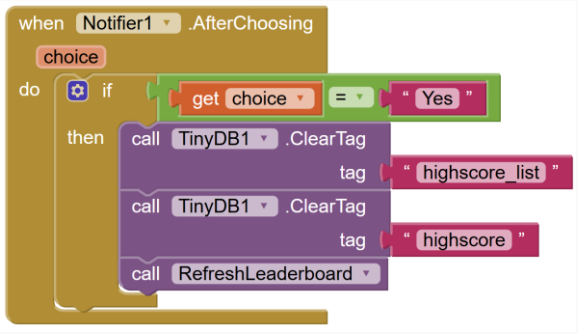


Procedure: RefreshLeaderboard

Reads the highscore_list tag from TinyDB (creating an empty list if none exists yet). For each of the 5 rank positions, it checks whether the list is long enough and the entry at that index is not empty. If a score exists, the label is set to "N. [score]". If no score exists for that position, it shows "N. —" to indicate an empty slot.

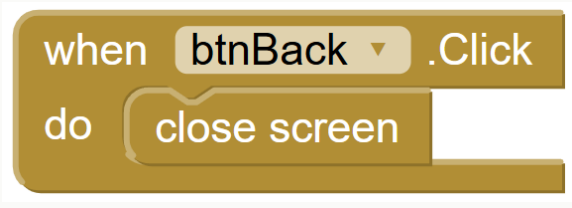
btnClear.Click

Triggers a confirmation dialog via Notifier1 asking "Are you sure you want to permanently wipe all high scores?" with Yes and No buttons (cancelable is set to false so the player must choose one).



Notifier1.AfterChoosing

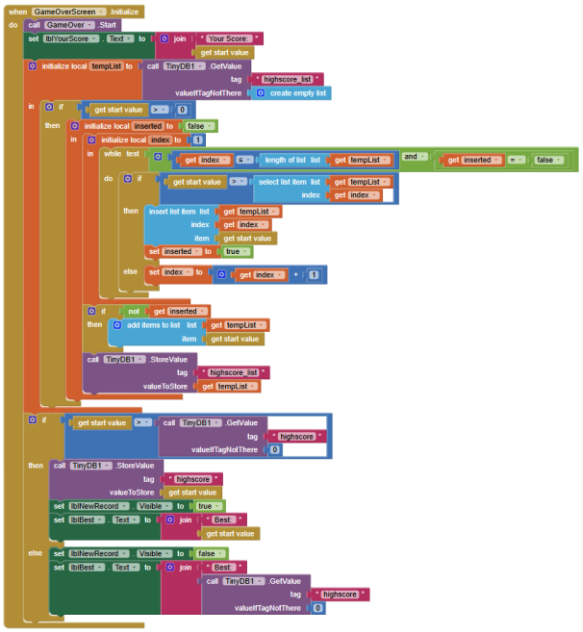
Fires after the player answers the clear dialog. If they chose "Yes": the highscore_list tag and the highscore (all-time best) tag are both cleared from TinyDB, and RefreshLeaderboard is called to reset all five label slots to "—".



btnBack.Click

Closes the HighScoresScreen and returns to the Main Menu.

GameOverScreen

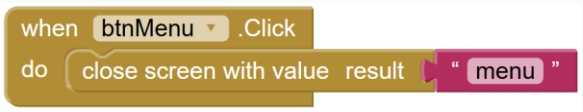


GameOverScreen.Initialize

On load: starts the game-over sound and displays the player's final score. It then reads the existing highscore_list from TinyDB. If the score is greater than 0, an insertion sort loop walks through the stored list and inserts the new score at the correct position, keeping the list in descending order. The updated list is saved back to TinyDB. Separately, if the new score beats the stored all-time best (highscore tag), the tag is updated and the "New Record!" label is made visible. Otherwise the label is hidden and the previous best score is shown.

btnPlayAgain.Click

Closes the GameOverScreen and passes the string "restart" back as the result value. The GameScreen's OtherScreenClosed handler receives this and immediately resets all state variables to start a fresh game without returning to the Main Menu.



btnMenu.Click

Closes the GameOverScreen and passes the string "menu" back as the result value. The GameScreen's OtherScreenClosed handler receives this and closes itself, returning the player to the Main Menu.

6. ABOUT THE DEVELOPER

Full name
Adam Iskandar bin Nazri

Student ID
2024277906

Programme
Diploma in Computer Science

Institution
UiTM Jasin

Course
ISP250 — Introduction to Software Programming

Semester
Semester 4

Year
2026